

TUẦN 4 (socket)

Sinh viên thực hiện các ví dụ sau:

TCP

Ví dụ chương trình TCPEchoClient sẽ kết nối đến TCPEchoServer ở chế độ kết nối, lần lượt gửi đến TCPEchoServer 10 ký tự từ '0' đến '9', chờ nhận kết quả trả về và hiển thị chúng ra màn hình.

```
package TCPsocket;
import java.io.*;
import java.net.Socket;
public class TCPEchoClient {
    public final static String serverIP = "127.0.0.1";
    public final static int serverPort = 7;
    public static void main(String[] args) throws InterruptedException,
    IOException {
        Socket s = null;
        try {
            s = new Socket(serverIP, serverPort);
            System.out.println("Client đã được tạo");
            InputStream is = s.getInputStream();
            OutputStream os = s.getOutputStream();
            for(int i='0'; i<='9'; i++) { //gửi '0'-'9' đến server
                os.write(i);
                int ch = is.read();
                System.out.println((char)ch);
                Thread.sleep(2000);
            }
        } catch (IOException ie) {
            System.out.println("Error: Can NOT create socket");
        } finally {
            if (s != null) {
                s.close();
            }
        }
    }
}
```

Ví dụ chương trình TCPEchoServer phục vụ tuần tự

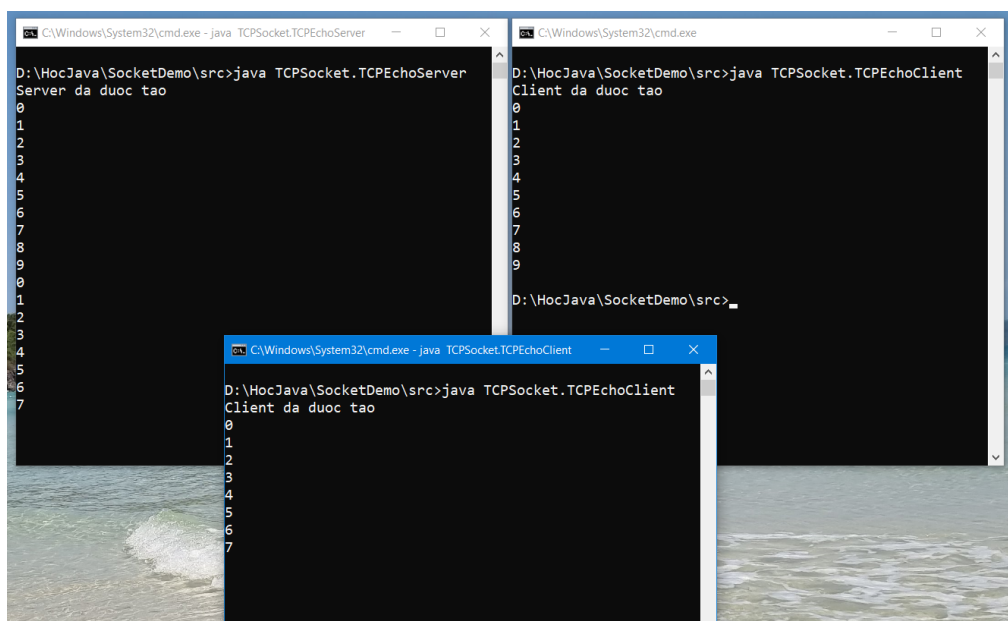
```
package TCPSocket;

import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;

public class TCPEchoServer {
    public final static int serverPort = 7;
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(serverPort);
            System.out.println("Server da duoc tao");
            while(true){
                try {
                    Socket s = ss.accept();
                    OutputStream os = s.getOutputStream();
                    InputStream is = s.getInputStream();

                    int ch = 0;
                    while(true) {
                        ch = is.read();
                        if(ch == -1) break;
                        System.out.println((char)ch);
                        os.write(ch);
                    }
                    s.close();
                } catch (IOException ie1) {
                    System.out.println("Connection Error: "+ie1);
                }
            }
        } catch (IOException ie) {
            System.out.println("Server Creation Error: "+ie);
        }
    }
}
```

Biên dịch và chạy chương trình: Mở một cửa sổ cmd chạy chương trình TCPEchoServer để khởi tạo server. Sau đó mở thêm 2 cửa sổ cmd để chạy chương trình TCPEchoClient để kết nối và gửi yêu cầu tới server. Kết quả cho thấy tại một thời điểm TCPEchoServer chỉ có thể phục vụ một client.



Ví dụ chương trình TCPEchoServer1 phục vụ song song

```
package TCPSocket;

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.ServerSocket;
import java.net.Socket;

public class TCPEchoServer1 {

    public final static int serverPort = 7;
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(serverPort);
            System.out.println("Server da duoc tao");
            while(true){
                try {
                    Socket s = ss.accept();
                    RequestProcessing rp = new RequestProcessing(s);
                    rp.start();
                    //
                }catch(IOException ie1) {
                    System.out.println("Connection Error: "+ie1);
                }
            }
        }catch(IOException ie) {
            System.out.println("Server Creation Error: "+ie);
        }
    }
}
```

```

package TCPSocket;

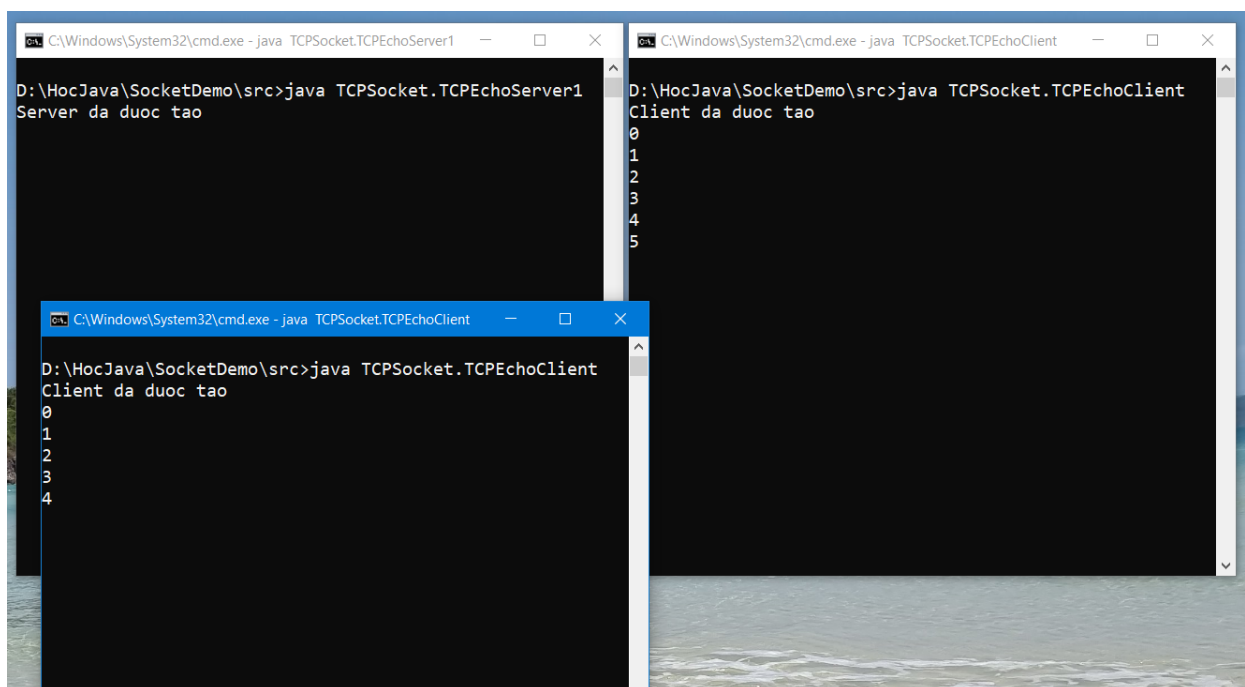
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;

public class RequestProcessing extends Thread{
    Socket channel; //socket của kênh ao nói với client hiện tại
    public RequestProcessing(Socket s) {
        channel = s;    //nhận socket của kênh ao nói với client
    }
    public void run() {
        try{
            OutputStream os = channel.getOutputStream();
            InputStream is = channel.getInputStream();
            while(true) {
                int n = is.read();
                if(n== -1) break;
                os.write(n);
                //
            }
        }catch(IOException ie) {
            System.out.println("Request Processing Error: "+ ie);
        }
    }
}

```

Biên dịch và thực hiện chương trình: Mở một cửa sổ cmd chạy chương trình TCPEchoServer1 để khởi tạo server. Sau đó mở thêm 2 cửa sổ cmd để chạy chương trình TCPEchoClient để kết nối và gửi yêu

cầu tới server. Kết quả cho thấy tại một thời điểm TCPEchoServer1 có khả năng phục vụ đồng thời nhiều client.



UDP

3.2.1. Xây dựng chương trình Server ở chế độ không kết nối

```
package UDPSocket;

import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;

public class UDPEchoServer {
    public final static int serverPort = 7;
    public static void main(String[] args) {
        try {
            DatagramSocket ds = new DatagramSocket(serverPort);
            System.out.println("Server created...");
            byte[] buffer = new byte[6000];
            while(true) {
                DatagramPacket incoming = new DatagramPacket(buffer,
buffer.length);
                ds.receive(incoming); //cho`nhân gói tin gói đến
//Lay du lieu khoi gói tin nhận và đổi tu mảng byte
ra chuỗi
                String theString = new String(incoming.getData(),0,i
ncoming.getLength());
                //tao gói tin chứa du lieu vừa nhận được
                DatagramPacket outSending = new DatagramPacket(theSt
ring.getBytes(),
                    incoming.getLength(), incoming.getAddress(),
incoming.getPort());
                ds.send(outSending);
            }
        } catch (IOException ie) {
            System.out.println(ie);
        }
    }
}
```

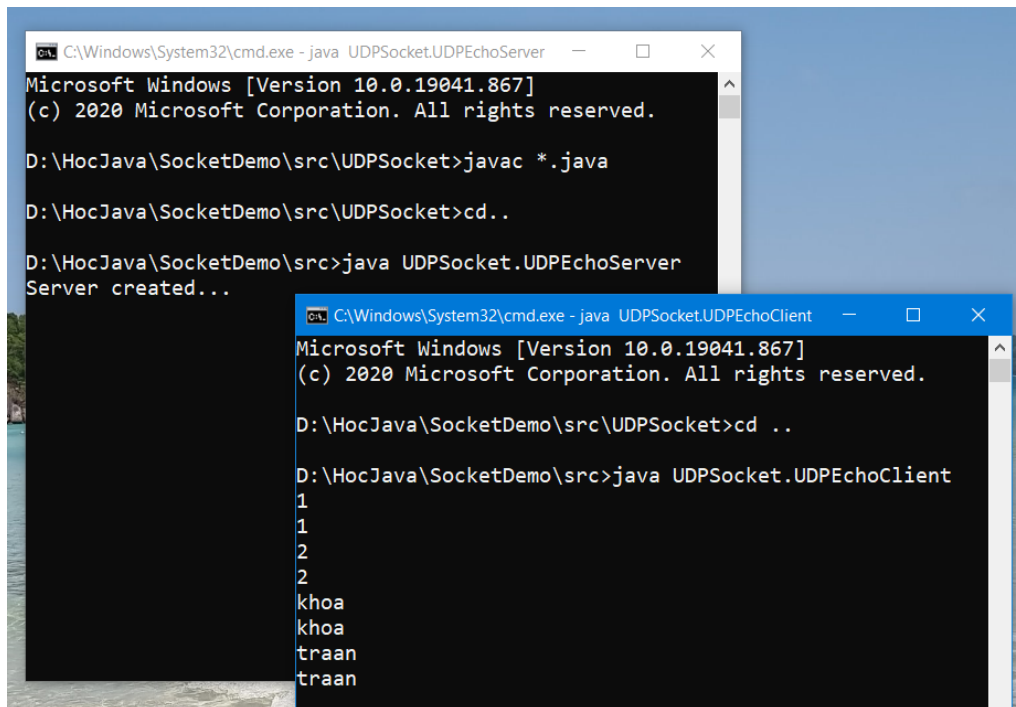
3.2.2. Xây dựng chương trình Client ở chế độ không kết nối

```
package UDPSocket;

public class UDPEchoClient {
    public final static int serverPort = 7;
    public static void main(String[] args) {
        try
        {
            DatagramSocket ds = new DatagramSocket();

            InetAddress server = InetAddress.getByName("localhost");
            while(true) {
                InputStreamReader isr = new InputStreamReader(System
.in);
                BufferedReader br = new BufferedReader(isr);
                String theString = br.readLine();
                byte[] data = theString.getBytes(); //doi chuoai ra m
ang byte
                //tao goi tin goi
                DatagramPacket dp = new DatagramPacket(data, data.le
ngth, server, serverPort);
                ds.send(dp);
                byte[] buffer = new byte[6000]; //vung dem cho du li
eu nhan
                //goi tin nhan
                DatagramPacket incoming = new DatagramPacket(buffer,
buffer.length);
                ds.receive(incoming);
                //doi du lieu nhan duoc dang mang bytes sang chuoai v
a in ra man hinh
                System.out.println(new String(incoming.getData(), 0,
incoming.getLength()));
            }
        }catch(IOException ie) {
            System.out.println(ie);
        }
    }
}
```

Biên dịch và chạy chương trình: Mở một cửa sổ cmd chạy chương trình UDPEchoServer để khởi tạo server. Sau đó mở thêm cửa sổ cmd để chạy chương trình UDPEchoClient để kết nối và gửi yêu cầu tới server.



```
C:\Windows\System32\cmd.exe - java UDPSocket.UDPEchoServer
Microsoft Windows [Version 10.0.19041.867]
(c) 2020 Microsoft Corporation. All rights reserved.

D:\HocJava\SocketDemo\src\UDPSocket>javac *.java

D:\HocJava\SocketDemo\src\UDPSocket>cd ..

D:\HocJava\SocketDemo\src>java UDPSocket.UDPEchoServer
Server created...

C:\Windows\System32\cmd.exe - java UDPSocket.UDPEchoClient
Microsoft Windows [Version 10.0.19041.867]
(c) 2020 Microsoft Corporation. All rights reserved.

D:\HocJava\SocketDemo\src\UDPSocket>cd ..

D:\HocJava\SocketDemo\src>java UDPSocket.UDPEchoClient
1
1
2
2
khoa
khoa
traan
traan
```

BÀI TẬP THỰC HÀNH

Bài 1:

Viết chương trình theo mô hình Client-Server sử dụng dụng **Socket ở chế độ có nối kết**. Trong đó :

+ Server làm nhiệm vụ đọc một ký tự số từ '0' đến '9'.

(Ví dụ : nhận số 0 : trả về "không" , 1 : trả về "một" ; 9 : trả về "chín", nếu nhận ký tự khác số thì trả về "Không phải số nguyên").

+ Client sẽ nhập vào 1 ký tự, gửi qua Server, nhận kết quả trả về từ Server và thể hiện lên màn hình

InputStream/OutputStream

DataInputStream/ DataOutputStream

Bài 2: (gửi nhiều tn cùng lúc + nhiều client)

Viết ứng dụng Chat đơn giản sử dụng Socket TCP.

Yêu cầu:

Xây dựng Server có thể lắng nghe kết nối từ Client.

Client sau khi kết nối thành công với Server có thể gửi tin nhắn (text) qua lại với Server

Bài 3:

Cải tiến bài trên với yêu cầu: Server và Client được cài đặt trên 2 máy khác nhau. Client sẽ xác định địa chỉ IP và Port dựa theo tham số truyền vào (args)

Bài 4:

Viết chương trình date/time client/server **TCP** theo mô tả sau

- Chương trình server cung cấp các chức năng sau: Date (xem ngày hệ thống), Time (xem giờ hệ thống), Date&Time (xem ngày giờ hệ thống). Các chức năng này có thể chọn qua một menu.
- Client:

1. Kết nối đến server: client nhập địa chỉ server cung cấp dịch vụ và port trước khi kết nối
 2. Nếu kết nối được thì nhập yêu cầu cần phục vụ:
 1. Time
 2. Date
 3. Date & Time
 3. Client nhận trả lời của server và in ra màn hình
- Server:
1. Chờ kết nối từ client tại IP/port đã đăng ký
 2. Cung cấp menu dịch vụ
 3. Xử lý yêu cầu và trả kết quả cho client
 4. (*) Cho phép nhiều client kết nối.

Bài 5:

Viết chương trình date/time client/server **UDP**

- Chương trình client cung cấp các chức năng sau:
1. Nhập địa chỉ server cung cấp dịch vụ và port
 2. Nhập yêu cầu cần phục vụ từ bàn phím và gửi đến server đã nhập:
 1. Time
 2. Date
 3. Date & Time
- Nhận trả lời của server và in ra màn hình

Bài 6:

Viết chương trình mô phỏng mô hình tính toán ở server(TCP):

- Server cung cấp các hàm tính toán, client gửi yêu cầu tính toán, sau đó gửi tham số (giá trị của n) đến server để nhận kết quả trả về.
- Các yêu cầu tính toán gửi từ client như sau:
1. Tổng $1+3+5+7+...+(2n+1)$
 2. Tổng $1*2 + 2*3+...+n*(n+1)$
 3. Biểu thức $1-2+3-4+...+(2n+1)$

Tương tự câu trên nhưng sử dụng giao thức UDP

Bài 7:

Viết chương trình gửi file (TCP)

Client :

1. Kết nối đến server có địa chỉ do người sử dụng nhập vào,
2. Người sử dụng nhập tên file cần truyền đến server
3. Người sử dụng nhập đường dẫn trên server để chứa file
4. Truyền file đến server

Tương tự câu trên, viết chương trình gửi file (UDP)

Bài 8:

Viết chương trình theo mô hình Client-Server sử dụng Socket ở chế độ có kết nối. Trong đó:

- Server sẽ nhận các yêu cầu là chuỗi có khuôn dạng sau:
- "OP Operant1 Operant2\n"**

Trong đó:

- OP là một ký tự chỉ phép toán muốn thực hiện: '+', '-', '*', '/'
- Operant1, Operant2 là đối số của phép toán
- Các thành phần trên cách nhau bởi 1 ký tự trắng ' '.
- Kết thúc yêu cầu bằng ký tự xuống dòng '\n'

Mỗi khi server nhận được một thông điệp nó sẽ thực hiện phép toán **Operant1 OP Operant2** để cho ra kết quả, sau đó đổi kết quả thành chuỗi và gửi về client

- Client cho phép người dùng nhập các phép toán muốn tính theo cách thức thông thường. Ví dụ: 100+200. Client tạo ra thông điệp yêu cầu theo đúng dạng do Server qui định, mô tả về phép toán muốn Server thực thi, rồi gửi sang Server, chờ nhận kết quả trả về và in ra màn hình.

Bài 9:

Viết chương trình Client-Server theo mô tả sau:

Server:

- Lưu trữ tập tin data.txt (để cho)
- Cho phép nhiều client có thể truy cập và đọc nội dung tập tin.

Client

- Kết nối đến server,
- Nhập tên file cần đọc nội dung.
- Hiển thị ra màn hình nội dung file do server trả về

Bài 10:

Viết chương trình Client-Server theo mô tả sau:

Server:

- Server có chức năng lưu tin nhắn của các client vào các tập tin riêng biệt (vd client1.txt, client2.txt,...)
- Thông báo bằng tin nhắn đến client khi việc lưu trữ thành công.
- Server cho phép nhiều client có thể truy cập và gửi tin nhắn cho mình.

Client:

- Client kết nối đến server
- Nhập các tin nhắn gửi đến server.
- Kết thúc tin nhắn bằng chuỗi "HET"

deb <http://http.kali.org/kali> kali-rolling main non-free contrib

deb <http://http.kali.org/kali> kali-last-snapshot main non-free contrib

deb <http://http.kali.org/kali> kali-experimental main non-free contrib

deb-src <http://http.kali.org/kali> kali-rolling main non-free contrib